

코드 설명 자료

개발 환경

- OS: "Debian GNU/Linux 12 (bookworm)"
- CPU: AMD Ryzen 9 5900X 12-Core Processor
- RAM: 64G
- GPU: NVIDIA GeForce RTX 3090
 - VRAM: 24576MiB
 - Driver Version: 570.133.07
 - CUDA Version: 12.8
- Python: 3.11.12
- 랜덤시드: 47

코드 실행

Preliminary

- 제출메일에 기재된 구글 드라이브 링크에는 모델 파일과 stanford 데이터셋, `open.zip` 이 있습니다
- stanford 데이터셋 관련 파일들은 `datasets/stanford_img_para_caption` 폴더에 unzip
- `open.zip` 은 eg 폴더에 zip 파일 그대로 넣기
- Model 폴더는 기존 Model 디렉토리 덮어쓰기 하면 됨
- 전체 파일 디렉토리 구조는 다음과 같습니다.

CURRENT_WORKING_DIR

```
├─ datasets
│   ├── Realworld/Realworld_aug
│   ├── stanford_img_para_caption
│   │   ├── stanford_df_rectified.csv
│   │   └── stanford_img
│   ├── visual7w
│   │   ├── dataset_v7w_telling.json
│   │   ├── visual7w_augmented_with_descriptions.jsonl
│   │   └── images
│   └── VMC/VMC_aug
├─ eg
│   ├── train.csv
│   ├── test.csv
│   ├── train_aug.jsonl
│   ├── sample_submission.csv
│   ├── train_input_images
│   └── test_input_images
└─ Model
    ├── flan-t5
    │   └── sota
    │       └── best
```

```

├── kosmos2
│   └── sota
│       ├── phase_1
│       └── phase_2
│           └── best
├── results
│   └── submission.csv
├── src
│   ├── __init__.py
│   ├── augment.py
│   ├── customdatasets.py
│   ├── inference.py
│   ├── init_setup.sh
│   ├── requirements.txt
│   ├── flan-t5
│   │   └── rep_t5.py
│   └── kosmos2
│       ├── __init__.py
│       └── rep_kosmos2.py

```

결과 재현을 위한 실행 순서

1. `src/init_setup.sh` 실행

필수 디펜던시(torch, cmake 등등)를 자동으로 설치해주고 huggingface 허브에 없는 Visual 7w 데이터셋을 다운로드 해줍니다

2. `src/augment.py` 실행

이 코드는 'LiAutoAD/Ristretto-3B' 라는 로컬모델을 다운받고, 이 모델을 통해 기존 데이터셋들에 이미지 캡셔닝을 추가하여 증강해줍니다.

3. `src/flan-t5/rep_t5.py` 과 `src/kosmos2/rep_kosmos2.py` 실행

각각 Flan-T5-Large와 KOSMOS-2 모델 훈련 코드입니다

두 스크립트는 `customdatasets` 모듈을 relative import 하기 위해 반드시 `python -m` 을 통해 실행해야 작동합니다

4. `src/inference.py` 실행

훈련된 모델을 통해 추론을 할 수 있는 코드입니다

추론이 완료되면 결과물인 `submission.csv` 는 results 폴더에 저장됩니다

제공된 모델 가중치를 통해 추론하고자 한다면 이것만 돌리면 됩니다

훈련을 새로 돌리면 모델 가중치가 덮어씌워집니다

만약의 경우..

transformers의 kosmos2 공식 모듈에는 에러가 있습니다. `rep_kosmos2.py` 에 몽키패치를 해두었으나, 혹시나 이에 관련해서 에러가 생길경우의 응급처치법입니다.

`transformers.models.kosmos2.modeling_kosmos2.Kosmos2TextTransformer.forward_embedding` 에 다음과 같이 한 줄을 추가해야 합니다.

```

def forward_embedding(
    self,
    input_ids,
    inputs_embeds: Optional[torch.Tensor] = None,
    image_embeds: Optional[torch.Tensor] = None,

```

```

        img_input_mask: Optional[torch.Tensor] = None,
        past_key_values_length: int = 0,
        position_ids: Optional[torch.Tensor] = None,
    ):
        # The argument `inputs_embeds` should be the one without being
        multiplied by `self.embed_scale`.
        if inputs_embeds is None:
            inputs_embeds = self.embed_tokens(input_ids)

        if image_embeds is not None:
            inputs_embeds = inputs_embeds.clone() # !!HOTFIX!! 이 줄을
추가해야 함
            inputs_embeds[img_input_mask.to(dtype=torch.bool)] =
image_embeds.to(inputs_embeds.device).view(
                -1, image_embeds.size(-1)
            )

        inputs_embeds = inputs_embeds * self.embed_scale

        # embed positions
        positions = self.embed_positions(
            input_ids=input_ids,
            inputs_embeds=inputs_embeds,
            past_key_values_length=past_key_values_length,
            position_ids=position_ids,
        )

        positions = positions.to(inputs_embeds.device)

        hidden_states = inputs_embeds + positions

        hidden_states = nn.functional.dropout(hidden_states,
        p=self.dropout, training=self.training)

        return hidden_states

```

이외에 다른 오류가 생길 경우 baeshstar@gmail.com 으로 연락부탁드립니다!
 감사합니다 😊

Appendix. requirements.txt

- requirements.txt 는 src 폴더에 들어가있습니다.
- 실행을 위한 최소한의 필수 패키지만으로 구성

```

accelerate==1.9.0
aiohappyeyeballs==2.6.1
aiohttp==3.12.15
aiosignal==1.4.0
annotated-types==0.7.0
attrs==25.3.0
bitsandbytes==0.46.1
certifi==2025.8.3
charset-normalizer==3.4.2

```

click==8.2.1
datasets==4.0.0
dill==0.3.8
evaluate==0.4.5
filelock==3.13.1
frozenlist==1.7.0
fsspec==2024.6.1
gitdb==4.0.12
GitPython==3.1.45
hf-xet==1.1.5
huggingface-hub==0.34.3
idna==3.10
Jinja2==3.1.4
joblib==1.5.1
MarkupSafe==2.1.5
mpmath==1.3.0
multidict==6.6.3
multiprocess==0.70.16
networkx==3.3
nltk==3.9.1
numpy==2.3.2
nvidia-cublas-cu12==12.8.3.14
nvidia-cuda-cupti-cu12==12.8.57
nvidia-cuda-nvrtc-cu12==12.8.61
nvidia-cuda-runtime-cu12==12.8.57
nvidia-cudnn-cu12==9.7.1.26
nvidia-cufft-cu12==11.3.3.41
nvidia-cufile-cu12==1.13.0.11
nvidia-curand-cu12==10.3.9.55
nvidia-cusolver-cu12==11.7.2.55
nvidia-cuspars-cu12==12.5.7.53
nvidia-cusparselt-cu12==0.6.3
nvidia-nccl-cu12==2.26.2
nvidia-nvjitlink-cu12==12.8.61
nvidia-nvtx-cu12==12.8.55
packaging==25.0
pandas==2.3.1
peft==0.17.0
pillow==11.3.0
platformdirs==4.3.8
propcache==0.3.2
protobuf==6.31.1
psutil==7.0.0
pyarrow==21.0.0
pydantic==2.11.7
pydantic_core==2.33.2
python-dateutil==2.9.0.post0
pytz==2025.2
PyYAML==6.0.2
regex==2025.7.34
requests==2.32.4
safetensors==0.5.3
sentencepiece==0.2.0
sentry-sdk==2.34.1

```
six==1.17.0
smmap==5.0.2
sympy==1.13.3
tokenizers==0.21.4
torch==2.7.1+cu128
torchaudio==2.7.1+cu128
torchvision==0.22.1+cu128
tqdm==4.67.1
transformers==4.54.1
triton==3.3.1
trl==0.20.0
typing-inspection==0.4.1
typing_extensions==4.12.2
tzdata==2025.2
urllib3==2.5.0
wandb==0.21.0
xxhash==3.5.0
yarl==1.20.1
```